

Rahmenbedingungen

- Dauer der Prüfung: 90 Minuten.
- Es sind keine Unterlagen erlaubt.

1. Felder

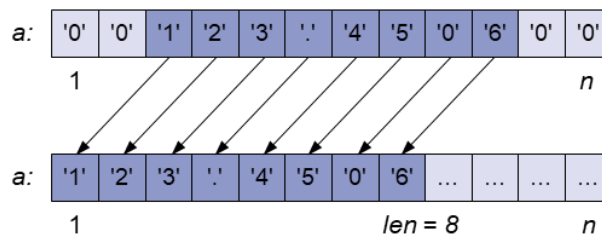
5 Punkte

Gegeben sind folgende Deklarationen für Felder mit n Elementen vom Typ *char*.

```
const
  n = ...
type
  CharArray = array [1:n] of char
```

Gesucht ist ein Algorithmus `Trim(↓↑a: CharArray ↑len: int)`, der im Feld a , das nur mit Ziffern und Dezimalpunkt befüllt ist, führende und nachgestellte Nullen entfernt und die Anzahl der verbleibenden Ziffern (inkl. Dezimalpunkt) len berechnet. Die Inhalte der nach dem Entfernen im Bereich von $len + 1$ bis n liegenden Elemente sind nicht von Bedeutung.

Beispiel:



2. Einfach-verkettete Listen

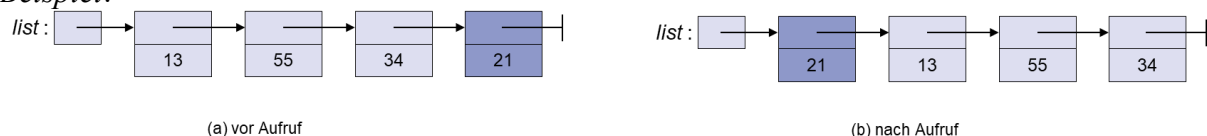
5 Punkte

Gegeben sind folgende Deklarationen für *einfach-verkettete Listen* von ganzen Zahlen (*int*).

```
type
  ListNodePtr = →ListNode
  ListNode = compound
    next: ListNodePtr
    data: int
  end -- compound
  ListPtr = ListNodePtr
```

Gesucht ist ein Algorithmus `MoveLastNodeToFront(↓↑list: ListPtr)`, der den letzten Knoten der Liste $list$ nach vorne bringt.

Beispiel:



3. Binärbaum

5 Punkte

Gegeben sind folgende Deklarationen für Binärbäume, die in den Knoten ganze Zahlen (*int*) enthalten.

```
type
  TreeNodePtr = →TreeNode
  TreeNode = compound
    left, right: TreeNodePtr
    data: int
  end -- compound
  TreePtr = TreeNodePtr
```

Gesucht ist ein rekursiver Algorithmus `MaxLeafValue(↓tree: TreePtr): int`, der den größten Wert (in der Komponente *data*) aller Blätter im Baum *tree* ermittelt. Für einen leeren Binärbaum muss der Algorithmus den Wert 0 liefern.

4. Komplexität von Algorithmen

3 Punkte

- (a) Was versteht man unter der *asymptotischen Laufzeitkomplexität*?
- (b) Welche asymptotische Laufzeitkomplexität hat das Suchen in einem binären Suchbaum, wenn er (1) perfekt (Dreiecksform) und wenn er (2) extrem degeneriert (d.h. lineare Liste) ist?
- (c) Erläutern Sie mit eigenen Worten, was man unter (1) der *Strukturkomplexität V* nach McCabe und (2) der *essentiellen Strukturkomplexität EV* nach McCabe versteht.

5. Rekursion und Iteration

2 + 1 Punkte

- (a) Wie muss ein rekursiver Algorithmus aufgebaut sein, damit man diesen ohne expliziten Kellerspeicher in einen iterativen Algorithmus transformieren kann?
- (b) Argumentieren Sie, ob Ihr rekursiver Algorithmus aus Aufgabe 3 ohne expliziten Kellerspeicher in einen iterativen Algorithmus transformiert werden kann.

6. Sortieralgorithmen

2 + 1 Punkte

- (a) In welchem Zusammenhang stehen der *ShellSort* (Sortieralgorithmus von D. L. Shell) und der *InsertionSort* (das Einfügesortieren) und wodurch drückt sich dieser aus?
- (b) Von welchen Faktoren hängt die Laufzeitkomplexität von *InsertionSort* ab?